

A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Tran Nguyen Viet Hoang¹ and Bui Ngoc Thai¹

Faculty of Computer Networks and Communications,
University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam
{23520541,23521412}@gm.uit.edu.vn

Abstract. Machine learning systems deployed in production can degrade when data distributions change, even if source code remains unchanged. This issue is critical for network intrusion detection, where traffic and attack behavior evolve over time. This paper does not propose a new intrusion detection classifier; instead, it presents a baseline drift-aware MLOps framework for user-imported and training-job-created tabular models, using Network Intrusion Detection Systems (NIDS) as a case study. The framework separates model governance from model serving through a control-plane/data-plane architecture and formalizes the lifecycle over tenants, models, inference requests, production windows, drift scores, and lifecycle actions. The prototype uses Django/FastAPI services, PostgreSQL, Redpanda, AWS Batch, object storage, optional MLflow support, and Evidently AI. It demonstrates how model artifacts can be onboarded, registered in a native model-evolution registry, served as APIs, monitored from production logs, and evaluated for Data Drift.

Keywords: MLOps · Data Drift · Multi-model Serving · Model Monitoring · Network Intrusion Detection

1 Introduction

Machine learning (ML) models are increasingly deployed as continuously running services rather than offline experimental artifacts. In a production setting, a model receives new input samples over time, generates predictions for downstream users or systems, and depends on monitoring mechanisms to remain trustworthy. Unlike traditional software services, an ML service may degrade even when its source code and infrastructure remain unchanged. One major cause is Data Drift, where the production data distribution differs from the reference distribution used during training, validation, or initial deployment.

This problem is especially relevant in Network Intrusion Detection Systems (NIDS). Network traffic is non-stationary: benign usage patterns change, infrastructure evolves, and attack strategies continuously adapt. A classifier trained on historical flow features may therefore become less reliable when deployed in a new environment or under a new attack distribution. A production-grade NIDS

should not only expose a prediction API but also record production data, compare it with reference data, and trigger lifecycle actions when the monitored distribution changes.

Existing MLOps research provides architectural concepts for automating ML development, deployment, and monitoring [1]. Other works discuss MLOps maturity, pipeline platforms, data quality, and continual model maintenance [2, 4, 5]. However, practical systems often treat monitoring as a passive observability layer. In many prototypes, drift reports are generated after deployment, but the monitoring result is not explicitly modeled as a control signal for model lifecycle decisions.

This paper presents a baseline framework that connects multi-model serving with drift-aware monitoring. The goal is to describe a reusable MLOps framework, not to claim a new state-of-the-art NIDS classifier. In the proposed design, NIDS is used as a case study because it naturally exposes the need for drift-aware operation. The framework is intentionally scoped to tabular classification models that are either imported by users or produced by controlled training jobs and then converted into serving-compatible artifacts.

The main contributions of this paper are:

- We propose a tenant-aware and drift-aware MLOps framework that integrates model onboarding, dynamic serving, production logging, drift detection, and lifecycle adaptation triggers.
- We formalize the framework using tenants, model functions, reference data, production windows, feature-level drift indicators, dataset-level drift ratio, and an action-selection function.
- We implement a prototype platform and instantiate it with a Network Intrusion Detection case study to demonstrate the feasibility of the proposed lifecycle.

2 Related Work

2.1 MLOps Architectures

MLOps extends DevOps principles to ML systems by coordinating data engineering, experimentation, training, deployment, monitoring, and governance. Kreuzberger *et al.* define MLOps as a paradigm for reliably and efficiently developing and maintaining ML systems in production [1]. Their architecture emphasizes components such as data pipelines, training pipelines, model registries, serving infrastructure, and monitoring. This work provides the architectural foundation for our paper, but our focus is narrower: we operationalize monitoring as a drift-aware lifecycle signal in a multi-model serving platform.

John *et al.* propose a framework and maturity model for MLOps adoption [2]. Their perspective suggests that production ML systems can be evaluated by the degree of automation and integration across the lifecycle. Our framework follows this direction by combining onboarding, serving, logging, and monitoring in a single flow. However, instead of discussing maturity at a general organizational level, we express the lifecycle as a set of model-specific state transitions.

2.2 MLOps Platforms and Case Studies

Several studies present MLOps platforms through implementation-oriented case studies. Zhou *et al.* discuss a machine learning pipeline platform that connects different stages of model development and deployment [3]. Subramanya *et al.* describe an MLOps workflow and apply it to electricity market forecasting [4]. These works show that case studies are useful for validating MLOps workflows in a concrete domain. Similarly, this paper uses NIDS as a case study, but the main artifact is the reusable serving and monitoring framework rather than the intrusion detection model itself.

2.3 Data Quality and Drift Monitoring

Production ML quality depends on the data observed after deployment. Renggli *et al.* argue that data quality should be treated as a first-class concern in MLOps [5]. Data Drift detection is one practical mechanism for identifying when production data no longer resembles the reference data. Gama *et al.* survey concept drift adaptation and show that non-stationarity is a central challenge in learning systems [6]. In this paper, Data Drift is used as an operational signal. Because production labels may be delayed or unavailable, the system first compares input distributions and then triggers inspection, retraining, or model replacement workflows when drift is detected.

2.4 Research Gap

The reviewed literature provides three useful foundations: general MLOps architecture, implementation-oriented MLOps case studies, and drift adaptation theory. The gap addressed in this baseline is the integration of these ideas into a concrete multi-model serving framework where drift detection is represented as a model-specific control signal. The proposed framework therefore bridges platform engineering and drift-aware model lifecycle management.

3 Problem Formulation

Let $\mathcal{U} = \{u_1, u_2, \dots, u_N\}$ denote the set of tenants using the platform. Each tenant $u \in \mathcal{U}$ owns a set of registered models:

$$\mathcal{M}_u = \{m_{u,1}, m_{u,2}, \dots, m_{u,K_u}\}. \quad (1)$$

Each model $m_{u,i}$ is represented by an inference function parameterized by θ :

$$\hat{y}_t = f_{\theta}^{u,i}(x_t), \quad (2)$$

where $x_t \in \mathbb{R}^d$ is the input vector at time t , d is the number of input features, and $\hat{y}_t$ is the prediction. For a tabular classification model, $\hat{y}_t$ may be a class label or a probability vector over class labels.

For every inference request, the platform records a production log:

$$z_t = (u, i, x_t, \hat{y}_t, c_t, t), \quad (3)$$

where c_t is the prediction confidence if available. The log stream associated with a tenant-model pair (u, i) forms the basis for monitoring.

Each deployed model has a reference dataset:

$$D_{\text{ref}}^{u,i} = \{x_1, x_2, \dots, x_n\}, \quad (4)$$

which represents the distribution observed during training, validation, or initial acceptance. At monitoring time T , the current production data window is:

$$D_{\text{cur}}^{u,i}(T) = \{x_t \mid T - W \leq t \leq T\}, \quad (5)$$

where W is the monitoring window size. The objective is not only to compute predictions but also to select a lifecycle action:

$$a_T = \pi(m_{u,i}, D_{\text{ref}}^{u,i}, D_{\text{cur}}^{u,i}(T), S_T), \quad (6)$$

where $\pi(\cdot)$ is the adaptation policy and S_T is the current lifecycle state of the model. Possible actions include no operation, alert generation, manual review, retraining trigger, model rollback, or model promotion.

For each feature j , the framework computes a drift score:

$$\delta_j = \Delta(P_{\text{ref}}(x_j), P_{\text{cur}}(x_j)), \quad (7)$$

where $\Delta(\cdot)$ is a statistical distance or hypothesis test between the reference and current feature distributions. A feature-level drift indicator is defined as:

$$I_j = \begin{cases} 1, & \delta_j > \tau_j, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where τ_j is the threshold for feature j . The dataset-level drift ratio is:

$$\rho = \frac{1}{d} \sum_{j=1}^d I_j. \quad (9)$$

The final drift decision is:

$$\text{Drift}(D_{\text{ref}}, D_{\text{cur}}) = \begin{cases} 1, & \rho \geq \tau_D, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

where τ_D is the dataset-level threshold. This formulation separates model prediction from monitoring and makes Data Drift an explicit input to lifecycle decisions.

4 Proposed Framework

The logical flow of the proposed framework consists of four stages. First, a user onboards a model artifact or registers an artifact produced by a controlled training job through the control plane. Second, the data plane exposes the model as a prediction endpoint. Third, each prediction produces an asynchronous log event. Fourth, the monitoring layer periodically reconstructs the current production window and compares it with the reference data. If drift is detected, the adaptation layer creates an alert or triggers a retraining workflow.

4.1 Control Plane

The control plane manages user-facing and governance operations. It stores tenants, authentication credentials, API keys, model metadata, artifact locations, endpoint URLs, access modes, and model statuses. In the prototype, this layer is implemented using Django. A model record contains the model name, description, owner tenant, artifact URI, access mode, generated endpoint, and lifecycle status.

When a model artifact is uploaded or registered from a completed training job, the control plane validates the available metadata, stores artifact references, and records the model in the native registry. The model can then be assigned an endpoint of the form `/models/{model_id}/predict`. This design decouples model ownership and metadata management from online inference.

4.2 Data Plane

The data plane is implemented as a generic FastAPI model server. Instead of deploying one service per model, the server resolves the model through the path parameter `model_id`. It reads the corresponding model metadata, loads the model artifact through MLflow pyfunc, validates the request payload against the expected input format when possible, and returns the prediction result.

Tenant-aware access control is enforced before inference. Public models can be invoked directly, while private models require a valid API key or bearer token. The resolved tenant identifier must match the model owner. This mechanism supports a platform setting where multiple users upload and serve independent models through a shared serving layer.

4.3 Production Logging Layer

Online inference should not depend synchronously on database writes. Therefore, the framework decouples prediction from logging through a message broker. After producing a prediction, the model server emits a production log event to Redpanda. A consumer service batches log events and persists them to PostgreSQL. Each log stores the tenant identifier, model identifier, timestamp, feature payload, and prediction result.

This design has two advantages. First, it keeps monitoring writes outside the synchronous prediction path. Second, it creates a unified production data source that can be queried per tenant and per model for drift detection, audit, and evaluation.

4.4 Drift Monitoring Layer

The monitoring layer compares reference data with recent production data. In the prototype, Evidently AI is used to compute feature-level drift and dataset-level drift summaries. Reference data can be stored as object storage artifacts, while production data are reconstructed from PostgreSQL logs. Drift reports can be exported as HTML or JSON artifacts for inspection.

4.5 Adaptation Layer

The adaptation layer maps monitoring outputs to lifecycle actions. In the baseline implementation, the primary action is a drift alert or workflow trigger. The framework can be extended to support retraining jobs, model evaluation gates, model promotion, rollback, and notification. This distinction is important: the current prototype demonstrates a drift-aware control loop, while fully autonomous retraining and deployment remain future work.

5 Drift-Aware Lifecycle Algorithm

The model lifecycle can be represented as a state transition system:

$$S \in \{\text{uploaded, validated, deployed, monitored, drifted, reviewing, retraining, promoted, failed}\}. \quad (11)$$

The transition function is:

$$S_{T+1} = \Phi(S_T, m_{u,i}, \rho, \tau_D, a_T), \quad (12)$$

where $\Phi(\cdot)$ updates the model state according to the drift ratio and the selected action. If $\rho < \tau_D$, the model remains in the monitored state. If $\rho \geq \tau_D$, the model is marked as drifted and the system can trigger alerting, review, or a workflow handoff.

6 Case Study: Network Intrusion Detection

The NIDS model is used as a case study to instantiate the proposed framework. A network flow is represented as a tabular feature vector $x \in \mathbb{R}^d$ containing statistics such as duration, packet counts, byte rates, and protocol-related attributes. The model predicts whether the observed traffic is benign or belongs to an attack class.

Require: Model package or training artifact A , metadata H , tenant u
Ensure: Registered model endpoint e

- 1: Validate package extension and archive structure
- 2: Validate serving metadata or record deployability status
- 3: Store or reference A in artifact storage
- 4: Create model record $m_{u,i}$ in the control plane
- 5: Assign lifecycle state $S \leftarrow \text{validated}$
- 6: Generate endpoint $e \leftarrow \text{/models/\{i\}/predict}$
- 7: Return endpoint e

Fig. 1. Model onboarding procedure.

Require: Input x_t , model identifier i , credential q
Ensure: Prediction $\hat{y}_t$ and optional lifecycle action a_T

- 1: Authenticate q and resolve tenant u
- 2: Retrieve model metadata $m_{u,i}$
- 3: Load or reuse cached model $f_{\theta}^{u,i}$
- 4: Compute prediction $\hat{y}_t \leftarrow f_{\theta}^{u,i}(x_t)$
- 5: Emit log event $z_t = (u, i, x_t, \hat{y}_t, t)$
- 6: **if** monitoring condition is satisfied **then**
- 7: Build production window $D_{\text{cur}}^{u,i}(T)$
- 8: **for** each feature $j = 1$ to d **do**
- 9: Compute drift score δ_j
- 10: Compute indicator I_j
- 11: **end for**
- 12: Compute drift ratio ρ
- 13: Select action $a_T \leftarrow \pi(m_{u,i}, D_{\text{ref}}, D_{\text{cur}}, S_T)$
- 14: **if** $\rho \geq \tau_D$ **then**
- 15: Trigger alert, review, or workflow handoff
- 16: **end if**
- 17: **end if**
- 18: Return $\hat{y}_t$

Fig. 2. Drift-aware inference and monitoring procedure.

In the current prototype, XGBoost is used as the classification algorithm because it is widely used for tabular data and can be packaged into a serving-compatible model artifact. The project contains multiple trained model versions, including binary-classification artifacts used for upload and model-evolution validation. These models are treated as artifacts and registry versions rather than hard-coded services. This allows the case study to demonstrate the general framework behavior: model registration, endpoint invocation, production logging, and drift monitoring.

NIDS is an appropriate case study for drift-aware MLOps for three reasons. First, network environments change over time, causing benign traffic distributions to shift. Second, attack patterns evolve, making historical training data incomplete. Third, labeled production data are often delayed or unavailable, so

input distribution monitoring can provide an early warning signal before supervised performance metrics are available.

7 Prototype Implementation

This section describes the implemented prototype as reflected in the repository. The purpose is to document the system boundary and implementation coverage, not to claim production readiness or performance characteristics.

7.1 Prototype Architecture

The prototype follows a control-plane/data-plane architecture. The control plane is implemented with Django and Django REST Framework and stores users, tenants, training jobs, model API records, registry versions, deployment metadata, and drift monitoring records in PostgreSQL. The data plane is implemented as a FastAPI model server that resolves a model endpoint, loads the serving artifact, performs prediction, and emits production log events. Redpanda is used as the asynchronous log transport, and a consumer service persists prediction records into PostgreSQL for later drift analysis.

The implementation also includes S3-compatible artifact storage, AWS Batch training jobs, a Python training runner, a model packager, MLflow integration, Evidently-based drift monitoring, and a Native Registry/Model Evolution layer. In the user-facing workflow, Native Registry data stored in PostgreSQL is the source of truth for Model Evolution. MLflow is treated as internal experiment lineage and model-format support rather than a required user interface.

Table 1. Prototype modules and responsibilities.

Module	Implementation	Responsibility
Control plane	Django/DRF	Manages tenants, jobs, registry, deployment, and drift APIs.
Training execution	AWS Batch + runner	Executes configured entry points and returns artifacts.
Artifact/registry layer	Object storage + PostgreSQL	Stores model artifacts, metadata, and version summaries.
Packaging/serving	Packager + FastAPI	Prepares or loads serving artifacts and exposes prediction endpoints.
Logging/drift	Redpanda + consumer + Evidently	Persists production logs and stores drift summaries.
Deployment configuration	Docker Compose/K8s/Argo files	Supports local integration and prototype/testbed deployment.

7.2 Training Job Execution

Training jobs can be created from an uploaded source package or from code stored through the Source Editor workflow. For the AWS Batch path, the backend prepares a single `source.zip` package and copies reference training data into a standard `train.csv` location when the source comes from S3 prefixes. The job definition then starts the training runner container with environment variables for the model output directory, training output directory, and training data channel. The user training script is expected to run as `train.py` or another configured entry point and to write model files under the model directory.

This training path does not require the user script to import MLflow or receive MLflow credentials. Instead, the user script writes normal model artifacts and optional metadata files, and the runner is responsible for packaging the output in a format the control plane can ingest.

7.3 Training Runner Output Contract

After training completes, the runner produces a required `model.tar.gz` archive and, when available, an `_mlops` metadata bundle containing training summary, metrics, parameters, model insights, artifact manifest, stdout/stderr logs, and warnings. Only the archive is required; the remaining metadata improves registry display and version comparison but does not invalidate the artifact when absent.

7.4 Metadata Ingestion and Native Registry

When a completed job is ingested, the control plane downloads the artifact, extracts metadata, and stores metrics, parameters, insights, manifest, and deployability summaries. These summaries are mirrored into Model Evolution when a model is registered from the job.

The Native Registry groups versions through `ModelFamily` and `ModelVersion` records linked to deployable `ModelAPI` records. Model Evolution displays tracking status, metrics, parameters, artifacts, model insights, deployability state, version comparison, deployment actions, drift summaries, and basic routing aliases.

7.5 Model Packaging and Serving

The prototype includes a model packager that prepares serving-compatible packages or images for supported artifacts. Build execution can be triggered through a local Docker path or through an Argo-oriented workflow path, depending on deployment configuration. The FastAPI model server loads the model artifact, including MLflow pyfunc artifacts where available, and exposes generated prediction endpoints. Prediction requests can be public or private depending on model configuration and are logged asynchronously for monitoring.

No latency or throughput claims are made in this paper. The current implementation demonstrates the integration path from registry metadata to serving endpoint and production log generation.

7.6 Drift Monitoring

Drift monitoring uses production records stored in PostgreSQL and reference data associated with a model. The Evidently integration computes feature-level and dataset-level drift summaries and stores report locations and drift result metadata in the control plane. The drift result can be exposed as a report link or summarized in Model Evolution.

In this baseline, drift is a monitoring and adaptation signal. It does not automatically redeploy a model, perform rollback, or select traffic weights. Such decisions require evaluation gates and human review policies that are outside the current baseline.

7.7 MLflow Tracking Integration

MLflow is used as optional internal tracking. After the control plane ingests the training artifact, it may log metrics, parameters, and metadata artifacts to an MLflow tracking server and store the run identifier. If MLflow tracking is disabled, Model Evolution still displays metrics, parameters, artifacts, model insights, and deployability information from the Native Registry. If MLflow logging fails, the metadata retained in PostgreSQL remains available for review and deployment decisions.

This design avoids requiring user training scripts to import MLflow. It also avoids making the MLflow UI the primary product workflow. The user-facing Model Evolution pages are backed by Native Registry records.

7.8 Deployment and Operational Scope

The repository contains Docker Compose configuration for local integration and Kubernetes-oriented manifests for a K3s-based prototype environment. It also includes CI/CD configuration for validation, container build, image publication, and GitOps-style deployment updates. These assets document deployment-oriented automation but are reported as prototype support rather than production hardening or evidence of the latest workflow success.

The AWS-facing boundary supports object storage and externalized training execution where configured. Conceptually, the web frontend calls the Django/DRF control plane for model upload, training, registry, deployment, and drift views; the control plane exchanges artifacts with object storage and submits training jobs; the runner returns artifacts for Native Registry ingestion; the FastAPI server handles inference and emits logs to Redpanda; the consumer stores logs in PostgreSQL; and the drift service writes summaries back to the control plane. Runtime evidence such as workflow logs, Kubernetes status, health checks, Batch traces, and screenshots should be attached separately before stronger operational claims.

The implemented baseline covers authentication, manual model upload, AWS Batch training, metadata ingestion, Model Evolution views, version comparison, build/deploy/smoke-test actions, production logging, drift report generation,

and a basic routing alias. Its limitations are intentionally explicit: autonomous re-training, automated promotion, canary or blue-green deployment, weighted traffic routing, resource quota enforcement, billing, and production-grade hardening are outside the completed baseline. K3s/Kubernetes deployment and CI/CD status should be reported with separate runtime evidence.

8 Evaluation Design

Because this paper is a baseline, evaluation focuses on feasibility and system behavior rather than NIDS accuracy. The validation plan covers five aspects: model onboarding, inference serving, production logging, drift detection, and operational workflow reachability. Onboarding checks whether model packages or completed training artifacts can be validated, stored, registered, and assigned endpoint metadata. Serving and logging checks whether tabular requests produce predictions and queryable production logs. Drift validation compares reference and production windows and reports drifted features, drift ratio, dataset-level decision, and report artifacts. Operational validation should use workflow logs, container image references, Kubernetes status, health checks, and end-to-end training or inference traces when runtime evidence is available.

9 Discussion

The proposed framework reframes MLOps monitoring as part of an adaptive control loop. The model server does not only return predictions; it also emits production evidence. The monitoring layer does not only generate reports; it produces a signal that can affect model lifecycle decisions. This design is useful for a SaaS or PaaS-style environment where different users upload different models but still need a common operational pipeline.

The current scope is intentionally limited. First, the system supports tabular classification models that can be represented as serving-compatible artifacts. Image, text, time-series, and large language models require additional schema validation and monitoring metrics. Second, arbitrary user-uploaded model artifacts introduce security risks. A hardened system must isolate execution, validate dependencies, restrict archive extraction, and enforce resource quotas. Third, autonomous retraining and promotion require reliable evaluation gates, rollback policies, and human approval mechanisms. Therefore, this baseline treats drift detection as an adaptation signal rather than an unconditional redeployment decision.

10 Conclusion

This paper presented a baseline design for a drift-aware MLOps framework for multi-model serving. The framework integrates tenant-aware model onboarding, training-job execution, native model-evolution metadata, dynamic endpoints,

asynchronous production logging, and reference-production drift monitoring. The system was formalized using tenants, model functions, production windows, drift indicators, lifecycle states, and adaptation actions. A Network Intrusion Detection model was used as a non-stationary security case study. Future work will focus on artifact-execution hardening, runtime deployment evidence, evaluation gates, and additional model types.

References

1. M. Kreuzberger, N. Kuhl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
2. M. M. John, H. H. Olsson, and J. Bosch, “Towards MLOps: A Framework and Maturity Model,” in *Proc. 47th Euromicro Conference on Software Engineering and Advanced Applications*, 2021, pp. 1–8.
3. Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *Proc. International Conference on Artificial Intelligence and Computer Engineering*, 2020, pp. 494–500.
4. S. G. Subramanya, S. Shetty, V. Patil, S. K. Gad, and S. N. Bhat, “From DevOps to MLOps: Overview and Application to Electricity Market Forecasting,” *Applied Sciences*, vol. 12, no. 19, 2022.
5. C. Renggli, B. Karlaš, B. Ding, F. Liu, K. Schawinski, W. Wu, and C. Zhang, “A Data Quality-Driven View of MLOps,” *arXiv preprint arXiv:2102.07750*, 2021.
6. J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A Survey on Concept Drift Adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
7. M. Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
8. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
9. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116.
10. B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.